

Arquitetura BigQuery e FinOps — CNPJ Lakehouse

1. Objetivo, escopo e princípios

Este documento descreve a evolução do Lakehouse CNPJ para GCP. O escopo da entrega executável permanece **local-first**: Prefect, dbt Core e DuckDB devem funcionar sem projeto GCP, credenciais ou serviços pagos. A proposta cloud é deliberadamente compatível com o mesmo contrato Bronze/Silver/Gold, para que a mudança de engine não altere chaves, regras de qualidade ou semântica dos marts.

O estado verificável de publicação fica no checkpoint de entrega do repositório.

Os princípios são: arquivos de origem imutáveis; processamento reexecutável e idempotente; menor leitura possível; dados e jobs rastreáveis; e custo observável antes, durante e depois da execução. O CNPJ básico é sempre texto de oito posições, incluindo zero à esquerda. Não há promessa de percentual de economia: o efeito de cada controle deve ser medido com bytes processados, slot-ms e custo real do projeto.



2. Organização de armazenamento e datasets

O bucket GCS deve estar na mesma região do BigQuery, por exemplo us-central1 ou southamerica-east1, escolhida antes da criação. A estrutura sugerida evita sobrescrever uma fonte e torna cada aquisição auditável:

```
gs://<project>-cnpj-bronze/  
  source_period=YYYYMM/  
    source_sha256=<sha256>/  
      Empresas0.zip ... Socios9.zip Simples.zip Cnaes.zip  
    manifest.json  
  samples/sample_id=<digest>/...csv.gz
```

O manifesto contém URL, checksum, tamanho, membro do ZIP, período, algoritmo de amostragem e contagens. O objeto ZIP original não é regravado; uma correção de fonte é outro prefixo de checksum. Uma regra de lifecycle pode mover ZIPs históricos para classe mais barata depois do período de retenção aprovado, preservando os manifestos. Uma expiração só é aplicável a cópias de trabalho e nunca à única cópia Bronze.

No BigQuery, criar datasets regionais separados: raw, silver, gold e snapshots. A identidade do Prefect possui apenas permissões de escrita em raw; dbt escreve Silver/Gold/snapshots; analistas recebem acesso de leitura seletivo em Gold. Aplicar IAM por dataset e, se houver campos sensíveis nos dados de sócios, políticas de coluna ou views autorizadas antes de ampliar o consumo.

Tabela	Partição	Cluster	Padrão de consulta e benefício
raw_companies, raw_establishments, raw_partners, raw_simples	DATE(ingested_at)	cnpj_basico	Reprocessamento por ingestão e reconciliação por empresa; reduz partições e blocos lidos.
raw_cnae	nenhuma	nenhuma	Domínio pequeno, lido integralmente; metadados de ingestão bastam para linhagem.
Silver grande	data de referência/ingestão somente quando usada em filtros	cnpj_basico	Transformações e joins por empresa; evita custo de uma partição sem predicado real.
dim_company_current	nenhuma	cnpj_basico apenas em volume que justifique	Busca pontual e joins por empresa; não fragmenta a dimensão atual.
dim_cnae, dim_cnae_hierarchy	nenhuma	nenhuma	Dimensões estáticas pequenas, lidas integralmente.
fct_company_activity_daily	reference_date com filtro obrigatório	uf, principal_cnae, cnpj_basico	Painéis por data, UF e atividade; combina pruning temporal e de blocos.
snapshot de Capital Social	DATE(dbt_valid_from)	cnpj_basico	Histórico por empresa e intervalo de vigência; limita scans temporais.
dim_company_capital_history	view lógica sobre o snapshot	herda cnpj_basico	Interface Gold de leitura histórica sem duplicar armazenamento.

3. Carga e modelagem

O fluxo Prefect primeiro envia e valida arquivos no GCS, registra o manifesto e só então lê/insere dados no BigQuery. Ele deve fazer streaming para disco/objeto, jamais reter ZIPs grandes em memória. A amostra local preserva relacionamentos: seleciona os 10 mil menores hashes de empresas a partir de todos os Establishments, depois retém registros correspondentes de Empresas, Sócios e Simples e a totalidade de CNAE.

Em produção full-volume, a mesma regra permite uma execução amostral ou a carga completa, sempre com `source_period`, `URL`, `arquivo/membro`, `número da linha`, `checksum`, `ingested_at` e `record_hash`. Uma chave física de linhagem é (`source_file_sha256`, `source_member`, `source_row_number`); registros idênticos de negócios não devem apagar a evidência da origem.

dbt normaliza CNPJ, converte Capital Social brasileiro com segurança, padroniza CNAE e deduplica na Silver. A Gold contém `dim_company_current`, `dim_cnae`, `dim_cnae_hierarchy`, `fct_company_activity_daily` e a visão de histórico de capital. O fato tem grão `reference_date + cnpj_basico`, somente para empresas com ao menos um estabelecimento de situação 02; as métricas incluem contagem de estabelecimentos ativos, sócios, capital, CNAE principal, UF e flags Simples/MEI.

O snapshot dbt usa `cnpj_basico` como chave e `capital_social` em `check_cols`. Assim, uma nova versão abre somente quando esse atributo muda. O modelo Gold histórico expõe início, fim e indicador de versão corrente.

4. Partições, clusters e incrementalidade

Partições devem ser escolhidas a partir do predicado que o consumidor realmente usa. `raw` é particionado por data de ingestão para permitir reprocessamento e retenção controlados. O fato é particionado por `reference_date`, pois painéis e cargas normalmente filtram uma data ou janela temporal. Em tabelas grandes e particionadas, configurar `require_partition_filter = true`; consultas operacionais que realmente precisem de toda a tabela devem ser exceções explícitas e revisadas.

O fato é clusterizado por `uf`, `principal_cnae` e `cnpj_basico`, nesta ordem, porque os filtros de segmento geográfico e atividade tendem a preceder a busca pontual pela empresa. Antes de fixar a ordem, validar com o histórico de consultas: se CNPJ virar o filtro dominante, a ordem deve ser reavaliada. Clustering não substitui filtro de partição, e campos de baixa seletividade não justificam clustering isolado.

As cargas incrementais `dbt/MERGE` devem restringir origem e destino à partição do `reference_date` ou do período de fonte. Um padrão é apagar/recarregar a partição lógica de uma execução idempotente, ou fazer `MERGE` usando a chave de negócio e predicado explícito de partição. Nunca usar `MERGE` sem filtro temporal em uma tabela grande, pois isso pode ler muitas partições. Para um backfill, receber uma lista/intervalo de partições aprovado, executar por lote pequeno e registrar bytes processados por lote.

5. Controles FinOps operacionais

1. **Prevenção na consulta.** Proibir SELECT * em modelos e análises recorrentes; selecionar somente as colunas necessárias. Exigir predicado de partição em fatos/raw. Usar estimativa de dry run e, para jobs de alto risco, maximumBytesBilled.
2. **Atribuição.** Todo job de pipeline recebe labels como application=cnpj_lakehouse, layer=raw|silver|gold, source_period, environment e owner. Jobs interativos só podem omiti-las sob exceção documentada, pois isso permite agrupar custo por pipeline, mês e camada na faturação e em INFORMATION_SCHEMA.JOBS.
3. **Orçamentos e alertas.** Criar budget mensal por projeto de dados com limites e notificações em patamares acordados (por exemplo, alerta antecipado e alerta crítico). Budget alerta, não interrompe consultas sozinho; controles de IAM, limites de bytes e revisão de jobs complementam a proteção.
4. **Retenção.** Configurar expiração para tabelas/intermediários transitórios e regra de lifecycle do GCS para cópias de trabalho. Tabelas Gold e Bronze canônica têm retenção definida pelo negócio, não uma expiração implícita.
5. **Monitoramento.** Publicar painel diário com total_bytes_processed, total_slot_ms, erros, duração, linhas afetadas, custo atribuído por label e partições lidas. Alertar aumentos anormais contra uma baseline por modelo/período, investigando mudança de plano, remoção de filtro ou duplicação de carga.

Exemplo de consulta de observabilidade, adaptando região e período:

```
select
  creation_time,
  labels,
  total_bytes_processed,
  total_slot_ms,
  state,
  error_result
from `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
where creation_time >= timestamp_sub(current_timestamp(), interval 7 day)
  and job_type = 'QUERY'
order by creation_time desc;
```

O indicador principal é bytes processados por execução e por modelo. A redução vem de ler apenas as partições/colunas e de o clustering restringir blocos relevantes; a magnitude depende da distribuição dos dados e dos filtros reais, portanto deve ser validada por job, sem porcentagens inventadas.

6. Resiliência, qualidade e onboarding

O pipeline trata erro de rede transitório com tentativas limitadas; arquivo inválido, schema divergente, resposta HTTP permanente e teste dbt falho encerram a execução com evidência. Downloads usam arquivo temporário até completar checksum. A gravação Bronze é transacional e o mesmo `sample_id` é um no-op em reexecução. Artefatos dbt (manifest, run results, stdout e stderr) ficam retidos por etapa.

As verificações incluem chaves únicas e não nulas, relações Establishments/Sócios → Empresas, relações de CNAE, valores permitidos de situação cadastral, Capital Social não negativo, grão único do fato e a regra de que o fato de empresa ativa não pode ter contagem ativa zero. A qualidade é gate de certificação: falha de teste impede promover ou expor a versão Gold candidata a consumidores, mas não presume rollback de tabelas físicas já escritas.

Para a entrega local, o avaliador começa pelo README e pelo runbook de análise local; GCP continua somente proposta. Onboarding cloud: provisionar projeto, região e budget; criar bucket/datasets/IAM; injetar credenciais por secret manager/identidade de workload, nunca em repositório; executar uma amostra controlada; validar artefatos e dashboard FinOps; e só então autorizar a carga completa. O runbook deve incluir reprocessamento de uma partição, revogação de acesso, recuperação do manifesto e contato responsável pelo custo.

Referências de produto: [particionamento e clustering](#), [controle de custo](#), [labels de jobs](#), [INFORMATION_SCHEMA.JOBS](#).